Cryptographie R309
Vigenère, ou la revanche d'Alice

14 septembre 2026

Lors de la séance précédente, nous avons vu que le chiffre de César ne résiste ni à la force brute ni à l'analyse fréquentielle. Le but de cette séance est d'étudier un chiffrement bien plus solide, dit *polyalphabétique*, qui utilise un mot secret plutôt qu'un simple décalage. Nous introduirons au passage un outil statistique très simple, l'indice de coïncidence, qui permet de reconnaître le type d'un chiffrement *sans en connaître la clé*. Nous verrons enfin que, malgré les apparences, ce nouveau chiffrement se ramène à plusieurs chiffres de César, ce qui permettra de le casser en réutilisant le code de la séance 1.

Rendu `vigenere.py`

Le code devra être réalisé sous `Python`. Les fonctions demandées (en rose dans l'énoncé) devront être sauvegardées dans un fichier nommé `vigenere.py`, à déposer sur le Moodle dans la boîte prévue. Le fichier ne doit contenir ni variable globale (sauf indication explicite) ni test. Les noms de fonctions et de variables spécifiés doivent être respectés.

Vous réutiliserez votre fichier `cesar.py` de la séance précédente, que vous placerez dans le même répertoire, en commençant `vigenere.py` par

```
from cesar import clean_message, cesar, F, J, break_cesar
```

Test `test-vigenere.py`

Un script de test est fourni afin de valider votre code au fur et à mesure du TP.

Fichiers fournis `message_bob.txt` ; `message_alice.txt` ; `frequences_francais.txt`

Exercice 1 Indice de coïncidence

Pour un texte m de longueur n écrit dans l'alphabet $\mathcal{A}$, on note n_x le nombre d'occurrences de la lettre x dans m. On appelle *indice de coïncidence* de m la quantité

$$IC(m) := \frac{\sum_{x \in \mathcal{A}} n_x(n_x - 1)}{n(n-1)}. \tag{1}$$

1. On tire au hasard deux lettres du texte m, à deux positions distinctes. Justifier que $IC(m)$ est exactement la probabilité que ces deux lettres soient identiques.

2. Calculer à la main $IC(m)$ pour $m =$ `ABRACADABRA`.

3. On suppose maintenant que m est un texte très long dans lequel chaque lettre apparaît avec la même fréquence $1/26$.
   a) Que vaut approximativement n_x en fonction de n ?
   b) En déduire que $IC(m) \simeq 1/26 \simeq 0.038$.

4. On note $\overline{F}(x)$ la fréquence de la lettre x dans un texte français, telle que donnée par le fichier `frequences_francais.txt`. Pour un texte français long, on admet que $n_x \simeq n\overline{F}(x)$.
   a) Montrer que $IC(m) \simeq \sum_{x \in \mathcal{A}} \overline{F}(x)^2$.
   b) On donne $\sum_x \overline{F}(x)^2 \simeq 0.078$. Comparer avec la valeur obtenue à la question précédente. Que peut-on en conclure ?

5. On chiffre le texte m par un chiffre de César de clé k, ce qui donne le texte m'.
   a) Comparer n_x dans m et n_{x+k} dans m'.
   b) En déduire que $IC(m') = IC(m)$.
   c) Un message codé a un indice de coïncidence de 0.041. A-t-il été chiffré par un chiffre de César ? Justifier. Insistons : on répond à cette question *sans* connaître la clé, et même sans avoir essayé de déchiffrer quoi que ce soit.

6. *(subsidiaire)* Bob envoie à Alice un message dans lequel le mot `DEMAIN` apparaît deux fois, aux positions 12 et 57 du message nettoyé. Les deux occurrences sont chiffrées de façon identique par le procédé de l'exercice 2 avec un mot-clé de longueur L.
   a) Que peut-on dire de $57 - 12$ par rapport à L ?
   b) Une autre répétition donne un écart de 30. Quelles sont les valeurs possibles de L ?

Exercice 2 Le chiffre de Vigenère

Contexte. Alice et Bob ont compris que leur enseignant lisait leurs messages. Alice propose donc un procédé attribué au diplomate Blaise de Vigenère : au lieu d'un décalage unique, on utilise un *mot-clé*. On écrit le mot-clé sous le message en le répétant autant de fois que nécessaire, puis on décale chaque lettre du clair du rang de la lettre de clé située juste en dessous.

Par exemple, avec le mot-clé `BOB` (c'est-à-dire les décalages 1, 14, 1) :

Clair	S	A	L	U	T	A	L	I	C	E
Clé	B	O	B	B	O	B	B	O	B	B
Décalage	1	14	1	1	14	1	1	14	1	1
Codé	T	O	M	V	H	B	M	W	D	F

Observons le point essentiel : la lettre `A` du clair, en 2e et en 6e position, est codée une fois par `O` et une fois par `B`. Une même lettre du clair ne donne donc plus toujours la même lettre codée, ce qui semble ruiner l'analyse fréquentielle. En revanche les deux `L`, qui tombent tous deux sous un `B` de la clé, donnent tous deux un `M` : c'est cette régularité résiduelle que nous exploiterons.

1. *(Chiffrement)* Écrire une fonction `vigenere` qui prend en argument un message `msg_clear` et un mot-clé `key_word`, et renvoie le message codé après avoir appliqué `clean_message` au message *et* au mot-clé. On vérifiera que

   ```
   vigenere("Salut Alice", "Bob")
   ```

   renvoie bien `"TOMVHBMWDF"`.

2. *(Déchiffrement)* Écrire une fonction `decode_vigenere` qui prend en argument un message codé `msg_coded` et un mot-clé `key_word`, et renvoie le message déchiffré. Le fichier `message_bob.txt` contient un message de Bob chiffré avec le mot-clé `BOB`. Que dit-il ?

Leur enseignant intercepte un long message d'Alice, dans le fichier `message_alice.txt`. Cette fois il ne connaît pas le mot-clé, ni même sa longueur. Voici comment il va procéder.

3. *(Indice de coïncidence)* Écrire une fonction `IC` qui prend en argument un message et renvoie son indice de coïncidence, tel que défini par la formule (1). Calculer l'indice du message d'Alice. Que peut-on en déduire ?

Supposons un instant que l'on connaisse la longueur L du mot-clé. Les lettres

du message situées aux positions $0, L, 2L, \ldots$ ont toutes été décalées de la *même* quantité, à savoir la première lettre de la clé. Il en va de même pour les positions $1, L+1, 2L+1, \ldots$, et ainsi de suite. Autrement dit :

en découpant le message en L colonnes, chaque colonne est un chiffre de César.

En Python, la i-ème colonne s'obtient très simplement par `msg_coded[i::L]`.

4. *(Longueur de la clé)* D'après la question 5 de l'exercice 1, chaque colonne a le même indice de coïncidence que le texte français dont elle provient, soit environ 0.078 — *mais seulement si L est la bonne longueur*. Sinon, la colonne mélange plusieurs décalages et son indice sera plus proche de 0.038.
   Écrire une fonction `key_length` qui prend en argument un message codé et un entier `lmax` (valant 20 par défaut), et qui pour chaque L de 1 à `lmax` calcule la moyenne des indices de coïncidence des L colonnes. La fonction renvoie la plus petite valeur de L pour laquelle cette moyenne dépasse 0.06.
   Afficher les valeurs obtenues pour le message d'Alice. Quelle est la longueur du mot-clé ? Pourquoi la fonction renvoie-t-elle la *plus petite* valeur convenable, et non celle qui maximise la moyenne ?

5. *(Décryptage)* Écrire une fonction `break_vigenere` qui prend en argument un message codé et renvoie le mot-clé ainsi que le message décodé. On procédera ainsi :
   - déterminer L à l'aide de `key_length` ;
   - pour chaque colonne, retrouver le décalage à l'aide de la fonction `break_cesar` écrite à la séance précédente ;
   - reconstituer le mot-clé à partir des L décalages, puis déchiffrer avec `decode_vigenere`.

   Appliquer au message d'Alice. Quel est le mot-clé ? Que raconte-t-elle à Bob ?

6. *(Conclusion)* Sans la moindre connaissance de la clé, nous avons cassé le chiffrement en testant 20 longueurs puis $L \times 26$ décalages. Combien de clés aurions-nous dû essayer par force brute pour un mot-clé de longueur 5 ? Et de longueur 10 ? Le chiffre de Vigenère est-il sûr ? Que faudrait-il changer pour qu'il le devienne ?

Quelques méthodes utiles en Python. `msg[i::L]` (découpage en colonnes) ; `ord` ; `chr` ; `enumerate` ; `min` avec l'argument `key`.